深圳大学 SHENZHEN UNIVERSITY

课程 COURSE

Big Data Analytics
大数据分析 [2026 国际商务]

Lecture 8 — Unsupervised Learning 第八讲·无监督学习
没有标签时，我们还能做什么

任课教师：戴谢尔 (Xieer Dai, David)
深圳大学 Shenzhen University

·第 6, 14 周·星期四 [11, 12] 节
上课地点：四方楼南 309
学分 2 选修课（讲座式）| 答疑：预约制 | E-mail: Xieer.Dai@szu.edu.cn

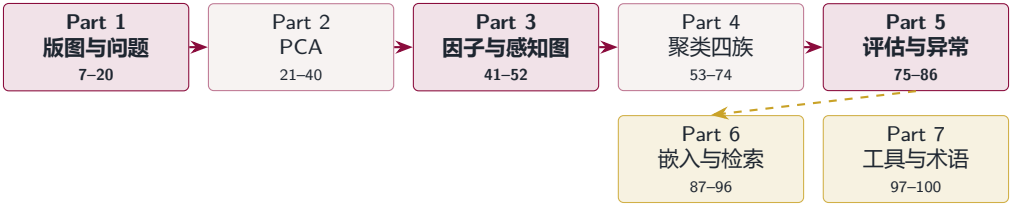
学习目标 Learning Objectives

完成本讲后，你应能够：

1. 解释 PCA 的几何含义与方差解释比例 explained variance ratio；
2. 区分主成分与因子，并写出 Fama–French 三因子模型；
3. 用四大家族的假设判断该用哪种聚类 clustering；
4. 区分内部指标与外部指标，并说出内部指标什么时候骗人；
5. 用孤立森林做异常检测、用支持度/置信度/提升度读关联规则；
6. 说清 t-SNE/UMAP 与近似最近邻 ANN 在 2026 工程里的位置。

来源：对应课程大纲第 8 讲目标：掌握降维、因子分析、聚类与异常检测的基本方法与适用条件。

本讲路线图 Roadmap（100 帧）



主线：版图 → 降维（PCA）→ 找潜变量（因子）→ 分组（聚类）→ 评估与异常 → 嵌入与检索

代码页 10 帧•样本页 16 帧；术语全程中英对照，代码逐行英文注释

与前一讲的衔接 With Labels and Without

L6–L7 的世界

每个样本都有 y：损失可算、指标可评、模型可调。一切以" 预测得准不准" 为裁判。

L8 的世界

没有 y，于是：

- 目标要自己定义（压缩？分组？找异常?)；
- 结果没有唯一答案；
- 只能用结构与解释来裁判。

这不是" 更难"，而是裁判换了：从" 预测对不对" 换成" 结构是否稳定、是否可解释、下游是否好用"。

来源：衔接说明为本讲组织；无监督评估的困境见帧 16。

本讲输出物 Deliverables

1) 选择地图

矩阵分解与聚类的两张菜单 + 一张"什么时候用哪个"的判据表（帧 10–14）

2) 可运行代码

14 个程序：PCA/因子/FF3/感知图/k-means/层次/DBSCAN/GMM/指标/异常/规则/嵌入/FAISS/主题

3) 术语表

72 条中英对照（CSV + Word/Excel + L6–L8 合并版）

所有数字都来自可复现的程序：`website/code/lecture08/`，种子固定 20260909，一条 `run_all.py` 重建 19 张图与 20 张表。

来源：程序与样本随课程仓库发布；术语表另附可打印版本。

全讲导览 Map of the Lecture

部分	帧	关键问题
版图与共同问题	7–20	无监督能做什么？为什么更难？怎么选方法？
主成分分析	21–40	主成分是什么？保留几个？为什么必须标准化？
因子与感知图	41–52	因子与主成分差在哪？三因子模型怎么读？感知图怎么做？
聚类四族	53–74	四种假设各适合什么形状？簇个数怎么定？
评估与异常	75–86	内部指标能信吗？异常怎么定义？规则怎么读？
嵌入与检索	87–96	图能当证据吗？向量库怎么选？主题模型怎么用？
工具与术语	97–100	用什么库？术语怎么背？

每个方法都走一遍直觉 → 目标函数 → 假设 → 样本图 → 代码 → 什么时候会骗你。

来源：导览为本讲安排；各部分均在后续帧展开。

无监督学习的四个任务 Four Jobs Without Labels

降维	聚类	异常检测	关联规则
把 p 维压到 k 维：主成分分析 principal component analysis (PCA)、t-SNE、UMAP。	把样本分组：k 均值 k-means、层次、密度、混合模型。	找出"不像其他样本"的点：孤立森林、LOF。	找"经常一起出现"的组合：Apriori、FP-Growth。

四个任务的输出完全不同：降维给坐标，聚类给标签，异常给分数，关联给规则。先明确要哪种输出，再选方法。

来源：四个任务的划分据 ESL 第 14 章与 scikit-learn 用户指南。

为什么无监督更难 Why It Is Harder

没有正确答案

同一份数据，k-means 给 4 类、层次聚类给 3 类、DBSCAN 给 2 类加一堆噪声——它们都没" 错"，因为" 类别" 本来就没人定义过。

只能靠三样东西

1. 结构：分组是否稳定、是否紧致；
2. 解释：分出来的组能不能说清楚；
3. 下游：分组对后续任务有没有用。

所以无监督项目的报告，不能只给一张图：必须写清" 我假设了什么样的结构、用什么指标检验、它支持了哪个业务结论"。

来源：评估困境据 ESL 与 ISLR 第 12 章；报告要求为本讲建议。

距离与相似度：一切的底座 Distance Is the Foundation

度量	形式	适用
欧氏 L_2	$\|x-z\|_2$	连续、同量纲；聚类默认
曼哈顿 L_1	$\|x-z\|_1$	对离群点更稳
余弦	$\frac{x^\top z}{\|x\|\|z\|}$	文本、稀疏向量
相关距离	$1-\rho(x,z)$	只看形状、忽略水平

两个必做的检查：(1) 标准化了吗？(2) 选的距离和业务含义一致吗？这两条决定了后面所有结论。

来源：距离度量据 scikit-learn 文档与 ESL；检查清单为本讲补充。

矩阵分解：一张菜单 Matrix Factorisation Menu

方法	约束	典型用途
PCA	正交、最大方差	降维、压缩、可视化
核 PCA kPCA	核映射后正交	非线性降维
截断 SVD / LSA	低秩近似	文本潜在语义、推荐
字典学习	稀疏编码	图像去噪、特征学习
因子分析 factor analysis	因子 + 误差	找潜在构念（帧 41）
ICA	统计独立	盲源分离（鸡尾酒会）
NMF	非负	主题、可加的部分-整体（帧 93）
LDA	生成式主题	文档主题（帧 93）

这张菜单来自课程讲义（scikit-learn 的 decomposition 模块目录）。共同点：都假设数据是低秩的；差别在约束与可解释性。

来源：菜单据课程讲义第 2 页与 scikit-learn 文档；说明为本讲补充。

分解方法怎么选 How to Choose a Factorisation

三个问题

1. 我要压缩还是解释?
2. 数据能假设什么（正交? 非负? 独立?）
3. 规模多大（能否放进内存）?

三条经验

- 只要压缩与降维 → PCA（或 Incremental PCA）;
- 变量非负、要" 部分构成整体"→ NMF;
- 文本主题 → LDA 或 NMF。

本讲只把 PCA 与因子分析讲透（帧 21–52），其余给出菜单与适用条件——它们都是同一套" 低秩 + 约束" 的思路。

来源：选择判据据 ESL 与 scikit-learn 文档；取舍为本讲安排。

聚类：一张菜单 Clustering Menu

家族	代表算法	核心假设
连通型 connectivity	层次聚类（单链/全链/平均/Ward）	近的连在一起
质心型 centroid	k 均值 k-means	簇是凸的、大小相近
分布型 distribution	高斯混合 Gaussian mixture、EM	簇是高斯的
密度型 density	DBSCAN、HDBSCAN、OPTICS	簇是高密度区域
其他	谱聚类、MeanShift、Affinity Propagation、BIRCH	图/模态/近邻

源稿第 3 页列了十种算法；本讲按四大家族归类，因为家族决定了"它能表示什么形状的簇"——这比记算法名重要得多。

来源：菜单据课程讲义第 3、19–23 页与 scikit-learn 文档；四族归类据 ESL。

四大家族的假设与后果 Assumptions and Consequences

家族	能表示	假设破了会怎样（帧 59、65、73）
连通型	任意形状	单链接会被" 桥" 粘住（帧 64）
质心型	凸、等大小	环状/悬殊大小被切成 Voronoi 块（帧 59）
分布型	椭圆高斯	非高斯形状被硬套成椭圆（帧 72）
密度型	任意形状 + 噪声	密度不均时参数难定（帧 73）

一句话：没有通用的聚类算法。选算法 = 选" 你认为簇长什么样"。这一句话能解释项目里 80% 的分群争议。

来源：假设与失败模式据课程讲义第 20–23 页；实测见后续样本页。

算法选择地图 The Selection Map

情形	首选	备注
凸簇、k 已知	k-means	快，需标准化
形状任意、有噪声	DBSCAN	两个参数
k 未知、要层次	层次聚类	$O(n^2)$ 内存
重叠、要概率	GMM	给软分配
文本/高维稀疏	谱聚类或嵌入 + 聚类	先降维

三条纪律

1. 先标准化，再看结果；
2. 用至少两个准则确定簇个数；
3. 报告稳定性（重抽样后是否一致）。

这张地图是项目报告里最值得抄走的一页：它把" 选算法" 变成" 回答三个关于数据形状的问题"。

来源：选择地图为本讲整理；纪律据评估困境（帧 75–78）。

无监督的”三件套”纪律 Three Habits

1) 标准化

不做标准化，距离与方差都会被量纲绑架 (帧 30 有实测对照)。

2) 定 K/k

用两个以上准则（肘部、轮廓、BIC)，并说明冲突时怎么取舍。

3) 报稳定性

重抽样后分群是否一致；不一致就别说" 发现了 X 个群体”。

这三条不是" 锦上添花"，而是无监督结论能不能站住的前提。它们对应评分表里" 结果与验证" 一栏。

来源：三条纪律据 L5 的评估协议与本讲实践；与评分表的对应为本讲补充。

评估的困境 The Evaluation Dilemma

指标类型	例子	局限
内部指标 internal metric	轮廓、CH、DB	会偏好凸簇；可被 K 操纵
外部指标 external metric	ARI、NMI、V-measure	需要真标签，业务里往往没有
下游任务	分群后做预测/营销的收益	最真实，但最慢

课程讲义把聚类评估单列成一节（2.3.11），因为它是无监督里最容易自欺的地方。本讲帧 77 会给出一个" 内部指标选错 K" 的实测例子。

来源：指标分类据 scikit-learn 的 clustering performance 文档与课程讲义。

感知图：无监督在营销里的经典用法 Perceptual Maps

做法

把品牌在两个可解释维度上打分（价格、性能、奢华—运动……），再用 PCA/MDS 把品牌放到平面上；圆的大小表示市场份额。

源稿的表述

营销者常用感知定位图 perceptual positioning map：每个圆的位置是品牌在两个维度上的感知，圆的大小是相对市场份额。

这是无监督里少见的"输出直接可汇报"的例子：一张图就能说清"我们的品牌离谁近、离谁远、往哪走"。本讲帧 55 会给完整做法。

来源：表述据课程讲义第 1 页备注与第 25–26 页；做法见帧 55。

本讲的三份贯穿样本 Three Running Samples

样本	维度	用在哪
手机参数表	8 列（CPU/GPU/核数/内存/相机/价格/份额）	PCA（帧 27–37）、因子分析（帧 41）
品牌感知	12 品牌 × 2 维度 × 260 受访者	感知图（帧 55）
收益面板	180 个月 × 三因子	Fama–French（帧 49）

全部合成数据（结构与源稿的示例一致，数值不是真实手机/品牌/行情），种子 20260909；图注会写明这一点。

来源：样本结构据课程讲义第 16 页的手机特征表；数据为本讲合成。

规模与工程 Scaling Considerations

数据放不进内存

增量 PCA incremental PCA（`partial_fit`）、稀疏矩阵、`np.memmap`；聚类用 MiniBatchKMeans。

样本太多

先抽样或用近似最近邻 approximate nearest neighbour (ANN)；本讲帧 90–92 会讲 FAISS 的召回-延迟权衡。

工程经验：降维往往是省钱的第一个动作——把 500 维压到 50 维，后面所有 kNN/聚类的成本都降一个量级。

来源：工程做法据 scikit-learn 文档；经验为本讲补充。

小结：先问”要结构还是要解释”

四个任务

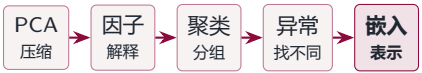
降维给坐标、聚类给标签、异常给分数、关联给规则。

三个习惯

标准化、双准则定 K、报稳定性。

接下来

- 压缩：PCA（帧 21–40）
- 潜变量：因子分析（帧 41–52）
- 分组：聚类四族（帧 53–74）



来源：小结为本讲整理。

直觉：把相关的变量压成几个不相关的主成分

源稿的" 复合变量"

一个复合变量 composite variable 是" 两个或多个统计上强相关的变量的组合"。PCA 就是找少数这样的复合变量，用它们概括原来的众多特征。

为什么可以压

如果 8 个手机参数其实只在" 性能" 和" 影像" 两个方向上变化，那么 2 个数字就够描述它们——信息本来就没那么多维。

一句话：PCA 找的是数据变化最大的那几个方向，并且要求这些方向互相垂直。前者决定" 压得准"，后者决定" 不重复"。

来源：" 复合变量" 的定义据课程讲义第 4 页；两条要求为本讲补充。

协方差矩阵与特征分解 Covariance and Eigen-decomposition

数学

中心化后算协方差矩阵

$$\Sigma = \frac{1}{n} X_c^\top X_c,$$

再做特征分解

$$\Sigma = V \Lambda V^\top,$$

V 的列就是特征向量 eigenvector，Λ 的对角是特征值 eigenvalue。

怎么读

- 特征向量 = 方向；
- 特征值 = 这个方向上的方差；
- 特征向量相互正交 $\rightarrow$ 主成分不相关。

实操上 sklearn 用 SVD 而不是显式求 Σ 的特征分解——数值更稳，但结果等价。所以你在代码里看不到 `eig`。

来源：推导据 ESL 第 14 章；SVD 与特征分解等价的事实用为本讲补充。

方差解释与投影 Variance and Projection

第一主成分

是使投影方差最大的单位方向：

$$v_1 = \arg\max_{\|v\|=1} \mathrm{Var}(Xv).$$

方差解释比例

$$\text{ratio}_k = \frac{\lambda_k}{\sum_j \lambda_j}.$$

前 k 个加起来就是" 保留下来的信息比例"。

投影公式：$z = V_k^\top x$，其中 V_k 是前 k 个特征向量组成的矩阵。用 k 个主成分表示原来的 p 维，误差是 $\sum_{j>k} \lambda_j$。

来源：优化形式与解释比例据 ESL 与 ISLR 第 12 章。

【代码】PCA：拟合、方差解释与重构

```python
from sklearn.decomposition import PCA, IncrementalPCA # batch and out-of-core PCA
from sklearn.preprocessing import StandardScaler      # standardisation before PCA
scaler = StandardScaler().fit(X)                      # fit the scaler on all rows
Xs = scaler.transform(X)                              # standardised matrix
pca_std = PCA().fit(Xs)                               # full PCA on the standardised data
ratio_std = pca_std.explained_variance_ratio_         # share of variance per component
pca_raw = PCA().fit(X)                                # full PCA on the raw scale
ratio_raw = pca_raw.explained_variance_ratio_         # share of variance per component
for k in [1, 2, 3, 4, 5]:                             # a few component counts
    ipca = IncrementalPCA(n_components=k, batch_size=10) # out-of-core PCA
    ipca.fit(Xs)                                      # fit in mini batches
    recon = ipca.inverse_transform(ipca.transform(Xs)) # reconstruct from k components
    incs.append({"components": k, "batch_rmse": float(np.sqrt(np.mean((Xs - recon) ** 2)))}) # the error
proj = pca_std.transform(Xs)[:, :2]                   # project on the first two components
```

实测（本讲手机样本，$p = 7$）：标准化后前两个成分解释 约 60% 的方差；用 5 个成分重构的 RMSE 是 0.12。完整程序 `l8_01_pca_basics.py`。

特征值就是”解释了多少方差”

课程讲义的说法

每个特征向量对应一个特征值；特征值给出该特征向量解释的总方差比例；PCA 按特征值从大到小排序——也就是按” 解释方差的用处” 排序。

等价说法

第一主成分解释最多，第二主成分在与第一正交的前提下解释剩余最多，依此类推。

注意：主成分的符号是任意的——把 v_1 换成 $-v_1$ 结果一样。所以报告里不要纠结” 第一主成分是正还是负”，要纠结它主要由哪些变量构成。

来源：原表述据课程讲义第 5 页；符号任意性为本讲补充。

碎石图怎么读 Reading a Scree Plot

三种定 k 的做法

1. 拐点：曲线从陡变平的位置；
2. 累计方差：到 80%–90% 为止；
3. 平行分析：与随机数据的特征值比较。

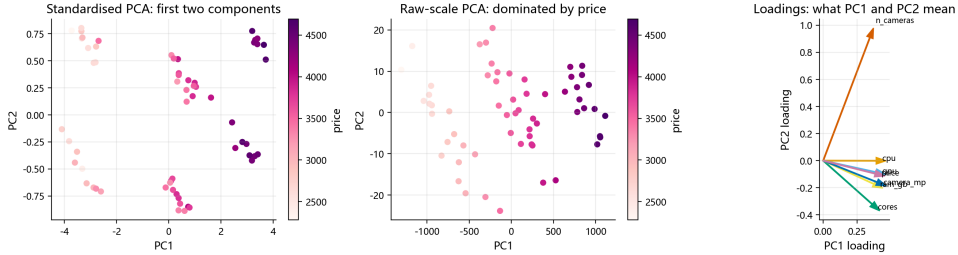
三个提醒

- 不要机械取 95%；
- k 应由下游任务决定（可视化用 2，建模可用 10+）；
- 用相关矩阵（标准化）时，特征值之和等于 p。

本讲实测（帧 33）：标准化后第一个成分 38%、前两个 60%、前四个 88%——这就是"保留几个"的经验依据。

来源：三种做法据 ISLR 与实践；实测数字来自程序 l8_01。

【样本 + 可视化】二维 PCA：手机的两个主成分



左：标准化后前两个主成分上的手机分布（颜色 = 价格），价格沿 PC1 单调上升；中：不做标准化时几乎只剩一个方向（被价格主导）；右：载荷箭头告诉你两个主成分分别由哪些参数构成——这正是" 游戏性能" 与" 影像能力" 的来源。

程序 18_01_pca_basics.py ·样本 18_smartphone.csv（合成）

主成分的解释要靠业务 Naming the Components

怎么读载荷

某个主成分在"CPU、GPU、核数" 上载荷都大，在" 相机" 上小——那就把它命名为性能；反之命名为影像。

三条纪律

1. 载荷要看相对大小，不看符号；
2. 一个成分最好只由一组相关变量主导；
3. 命名要能被业务同事复述。

源稿第 17 页的例子正是这样：CPU 频率与相机像素高度相关，于是" 两个原始变量" 被解释成" 游戏性能" 与" 拍照能力" 两个成分。

来源：源稿例子据课程讲义第 17 页；三条纪律为本讲补充。

标准化与否：PCA 最容易被忽略的一步

不标准化的后果

方差大的变量自动成为第一主成分——价格（以千元计）会压过核数（以个计），于是" 第一主成分" 其实只是" 价格"。

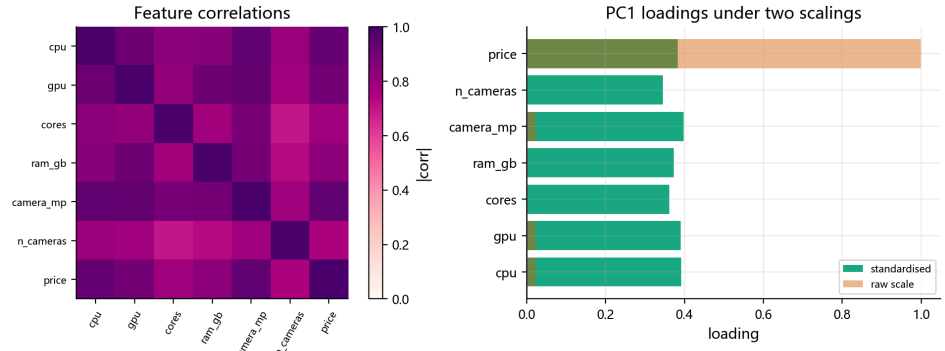
做法

用相关矩阵（等价于标准化）还是协方差矩阵，要在报告里写清楚；两者结论可能完全不同（帧 30 的实测）。

经验规则：变量含义与单位不同就一定标准化；只有" 同一物理量的多个测量" 时，才考虑保留原尺度。

来源：标准化对 PCA 的影响据 ISLR 第 12 章；经验规则为本讲补充。

【样本 + 可视化】标准化前后：主成分方向完全不同



左：七个参数的相关矩阵（CPU 与 GPU、价格与相机都高度相关）；右：同一个 PC1 在两种尺度下的载荷——标准化后它由" 性能组" 主导，不标准化时被" 价格" 独占。所以" 第一主成分是什么" 取决于你做没做标准化。

程序 18_01_pca_basics.py ·两种做法的方差解释见 18_pca_variance.csv

主成分个数的三种定法 Choosing the Number of Components

方法	判据	适合
碎石图拐点	曲线由陡变平	快速探索
累计方差	80% / 90% / 95%	压缩、去噪
平行分析	特征值大于随机数据的对应值	更严谨的统计依据
下游任务	用不同 k 跑后续模型比效果	最终决定

报告写法：给出碎石图 + 累计方差曲线 + 你选的 k 与理由。三个都给，评审才知道你不是随手取的。

来源：三种方法据 ISLR 与实践；报告写法为本讲建议。

PCA 在偏差-方差里扮演什么角色

降维 = 主动丢信息

丢掉 $p - k$ 个方向，就引入了偏差（这些方向上的结构没了），换来方差下降（参数少了、距离更稳）。

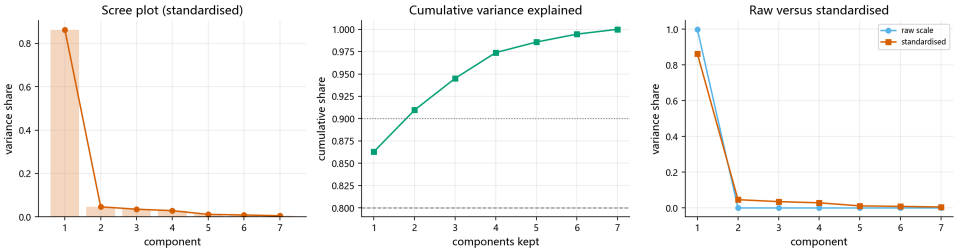
丢掉的是什么

低方差方向。它们可能是噪声（丢掉好），也可能是关键的小信号（丢掉坏）——这是 PCA 最大的风险。

所以" 用 PCA 降到 2 维做可视化" 和" 用 PCA 降到 10 维做建模" 是两件事：前者接受失真，后者要保住信号。

来源：偏差-方差视角据 ESL 第 14 章；两种用途的区分据 L5 的讨论。

【样本 + 可视化】碎石图与累计方差



左：标准化后的碎石图（第一个成分 38%，之后快速下降）；中：累计方差，虚线是 80% 与 90% 两条经验线——本样本 4 个成分到 88%；右：不标准化（原尺度）时第一成分独占 74%，几乎等于" 价格" 一个变量。

程序 18_01_pca_basics.py ·方差表 18_pca_variance.csv

PCA 的适用条件 When PCA Is Appropriate

适合

- 连续变量、线性结构为主；
- 变量之间有较强相关（否则压不动）；
- 数据近似对称、噪声近似各向同性。

不适合

- 强非线性（用核 PCA 或自编码器）；
- 类别变量（用 MCA 或嵌入）；
- 需要解释单变量效应（PCA 把变量混在一起了）。

一句判据：如果下游要"看某个变量的影响"，别用 PCA 当特征——它会把这个变量的系数摊到多个主成分上，解释不了。

来源：适用条件据 ISLR 与实践；判据为本讲补充。

PCA 在图像压缩里怎么用 PCA for Image Compression

做法

把图像切成 8×8 的小块，每块展成 64 维向量，在所有小块上做 PCA，得到 64 个" 图像基"；保留前 k 个基就能重建整张图。

为什么有效

自然图像的小块高度相关，少数基就够；压缩比 $= 64/k$（每个块只存 k 个系数）。

这是 JPEG 的思想来源之一（JPEG 用固定的 DCT 基，PCA 用从数据学出来的基）。课程讲义第 12–14 页给了 Mudrová & Procházka 的图像例子。

来源：做法与来源据课程讲义第 12–14 页；与 JPEG 的关系为本讲补充。

压缩比与重建误差 Compression Ratio and Error

两个指标

$$\mathrm{MSE} = \frac{1}{N}\sum_i (x_i - \hat{x}_i)^2, \qquad \mathrm{PSNR} = 10\log_{10}\frac{1}{\mathrm{MSE}}.$$

PSNR 越高越好，单位是 dB。

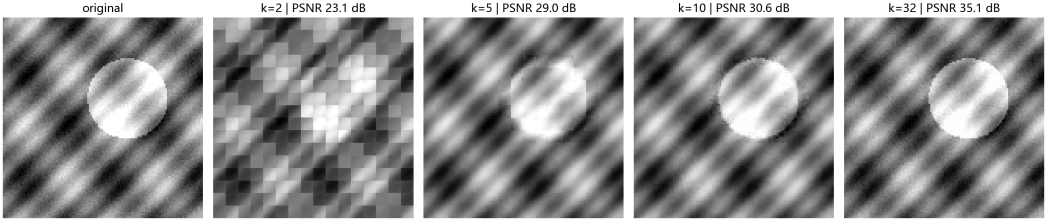
权衡

k 越大：误差越小、压缩比越低。报告里要给出" 在可接受的 PSNR 下能达到多少压缩比"。

本讲实测（帧 37）：$k = 2$ 时 PSNR 约 20 dB（勉强看出形状），$k = 10$ 时 30.6 dB，$k = 32$ 时 35.0 dB——误差随 k 快速下降后变缓。

来源：指标定义据图像处理通行做法；实测数字来自程序 l8_02。

【样本 + 可视化】图像压缩：k 从 2 到 32



合成图像切成 8×8 小块（共 256 块）：$k = 2$ 时只剩粗略明暗；$k = 5$ 轮廓出现；$k = 10$ 细节与纹理基本恢复（PSNR 30.6 dB）；$k = 32$ 几乎与原图一致（35.0 dB）。

程序 `l8_02_pca_images.py` ·数据 `l8_patches.npz` ·误差表 `l8_pca_image_errors.csv`

大规模数据：Incremental PCA 与 out-of-core

问题

普通 PCA 要一次性把全部数据放进内存；数据有几十 GB 时做不到。

解法

增量 PCA incremental PCA：分块 `partial_fit`，结果与批量 PCA 几乎一致（本讲实测差 7×10^{-5}）。

三种用法（课程讲义第 18 页）：(1) 从硬盘/网络顺序取块调 `partial_fit`；(2) 对稀疏矩阵或用 `np.memmap` 映射文件后调 `fit`；(3) 用随机化 SVD（`randomized_svd`）近似。

来源：用法据课程讲义第 18 页与 scikit-learn 文档；实测差来自程序 l8_02。

核 PCA：非线性降维 Kernel PCA

想法

与 L7 的核技巧同源：把样本隐式映射到高维，在高维里做 PCA，再投影回低维。线性做不了的结构就能被拉开。

代价

- 需要选核与核参数（RBF 的 γ）；
- 核矩阵 $n \times n$，大样本吃不消；
- 没有" 方差解释比例" 的干净解释。

实践顺序：先 PCA；如果二维图上明显" 缠绕" （同心环、螺旋），再试核 PCA 或 UMAP（帧 87）。

来源：核 PCA 据 scikit-learn 文档与 ESL；实践顺序为本讲建议。

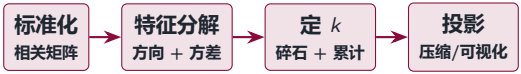
小结：PCA 是”线性压缩”的标准答案

四句话

主成分 = 方差最大的正交方向；特征值 = 解释的方差；标准化与否改变结论；k 由碎石图 + 累计方差 + 下游任务共同决定。

两个用途

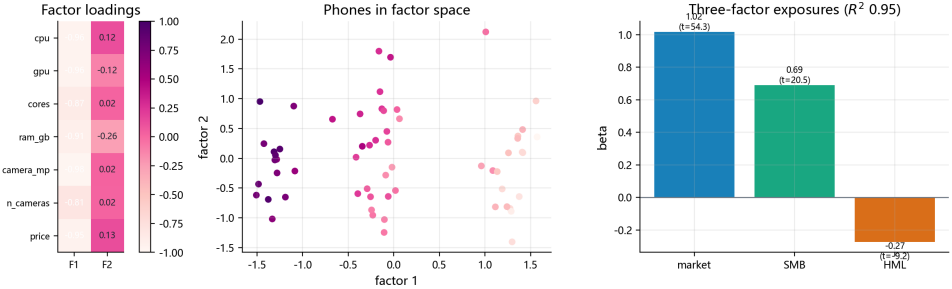
可视化 (2 维，接受失真) 与建模 (10–50 维，保住信号) ——不要混为一谈。



下一部分：如果主成分” 解释不了”，就需要因子——那是带误差项的模型，不是单纯的方差分解。

来源：小结为本讲整理。

【样本 + 可视化】因子载荷：手机参数背后的两个因子



左：七个参数在两个因子上的载荷——因子 1 由 CPU/GPU/核数/内存主导（" 性能"），因子 2 由相机与价格主导（" 影像/高端"）；中：手机在因子空间里的位置；右：FF3 回归的三个 β 与 t 值。

程序 `l8_03_factor_fama.py` ·载荷表 `l8_factor_loadings.csv` ·样本 `l8_smartphone.csv`

因子分析与 PCA：差在”误差项”

维度	主成分分析	因子分析
模型	无模型：X 的方差分解	有模型：$x = \Lambda f + \epsilon$
目的	压缩与降维	解释变量为什么相关
唯一性	主成分是 X 的确定性函数	因子有旋转不确定性（帧 44）
误差	只留" 被丢掉的方向"	显式建模唯一性方差 uniqueness

实践判据：要" 压缩数据" 用 PCA；要" 解释构念" 用因子分析。两者结果常常相似，但解释的语言不同。

来源：区别据课程讲义第 6 页与 ESL；判据为本讲补充。

六种因子提取方法 Six Extraction Methods

方法	特点（课程讲义第 7–8 页）
主成分法 PCA	做因子提取最常用；提取最大方差，直到无有意义的方差
典型因子 canonical(Rao)	用主轴法，求与观测变量典型相关最高的因子；对尺度变化不敏感
公因子法 common (PFA/PAF)	找最少的、能解释共同方差的因子
影像因子 image	基于" 用其他变量预测该变量" 的相关矩阵
Alpha 因子	最大化因子的可靠性；假设变量是从变量总体中随机抽的
因子回归模型	因子模型与回归模型的组合，因子部分已知

一句话：方法的选择取决于你用" 共同方差" 还是" 总方差"。实践里 PCA 与 PFA 最常见，差异在变量误差较大时明显。

来源：六种方法与描述据课程讲义第 7–8 页；总结为本讲补充。

旋转：让载荷可解释 Rotation

为什么转

因子解不唯一：把因子空间做一个正交变换，拟合优度不变，但载荷分布变了。转一转往往能让" 每个变量只在一个因子上有高载荷"。

两种转法

- 正交（Varimax）：因子保持不相关；
- 斜交（Promax/Oblimin）：允许因子相关，更贴近现实构念。

报告纪律：说明用了哪种旋转、为什么。不旋转的" 原始解" 往往难以命名，审稿人会追问。

来源：旋转的机制据课程讲义第 6 页与 ESL；纪律为本讲补充。

【代码】因子分析与三因子回归

```python
from sklearn.decomposition import FactorAnalysis      # maximum-likelihood factor analysis
X = StandardScaler().fit_transform(phones[features].values) # standardise before factoring
fa = FactorAnalysis(n_components=2, random_state=42).fit(X) # two latent factors
loadings = pd.DataFrame(fa.components_.T, index=features, columns=["F1", "F2"]) # loading matrix
scores = fa.transform(X)                              # factor scores per phone
mkt = RNG.normal(0.006, 0.045, n)                     # market excess return
smb = RNG.normal(0.002, 0.028, n)                     # small minus big
hml = RNG.normal(0.001, 0.030, n)                     # high minus low
ret = (0.0005 + beta_m * mkt + beta_s * smb + beta_h * hml # the true model
       + RNG.normal(0, 0.012, n))                     # idiosyncratic noise
Xm = sm.add_constant(panel[["mkt_excess", "smb", "hml"]].values) # design matrix with a constant
ols = sm.OLS(Y, Xm).fit()                             # the three-factor regression
ci = ols.conf_int()                                   # 95 per cent intervals
table = pd.DataFrame({"term": ["alpha", "beta_mkt", "beta_smb", "beta_hml"], # coefficient names
                      "coef": np.round(ols.params, 4),   # estimates
                      "t_value": np.round(ols.tvalues, 3)}) # t statistics
```

实测：因子 1 在 CPU/GPU/核数/内存上载荷高；FF3 回归还原出 $\beta_m = 1.05$、$\beta_{SMB} = 0.63$、$\beta_{HML} = -0.31$，R^2 0.959。完整程序 `18_03_factor_fama.py`。

Fama–French：为什么要在 CAPM 上加两个因子

CAPM 的不足

单因子（市场）解释不了的系统性差异：小盘股长期跑赢、价值股长期跑赢——这些不是噪声，而是规模与估值两个维度上的风险溢价。

做法

Fama & French（1992）在 CAPM 上加两个因子：SMB（小减大）与 HML（高账面市值比减低），形成三因子模型 three-factor model。

这是" 因子分析" 在金融里最著名的应用：从收益的协方差结构里提炼出少数共同因子，再用它们解释横截面差异。

来源：历史与动机据 Fama & French (1992, 1993) 与课程讲义第 9–10 页。

三因子模型的形式 The Three-Factor Model

公式

$$E(R_i) - R_f = \beta_m \big[E(R_m) - R_f\big] + \beta_s \cdot \mathrm{SMB} + \beta_h \cdot \mathrm{HML},$$

写成分期回归形式：

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_m \left(\mathrm{mkt}_t\right) + \beta_s \,\mathrm{SMB}_t + \beta_h \,\mathrm{HML}_t + \varepsilon_{i,t}.$$

三项的含义

- β_m：市场暴露（与 CAPM 同）；
- β_s：规模暴露（正 = 偏小盘）；
- β_h：价值暴露（正 = 偏价值）。

注意

β_m 与 CAPM 的 β " 类似但不相等"——因为另外两个因子也分担了一部分解释力（课程讲义原话）。

来源：公式与说明据课程讲义第 10 页；回归形式为本讲补充。

SMB 与 HML 的含义 What SMB and HML Measure

SMB

Small **M**inus **B**ig：小市值组合减大市值组合的历史超额收益。暴露为正 → 组合偏小盘。

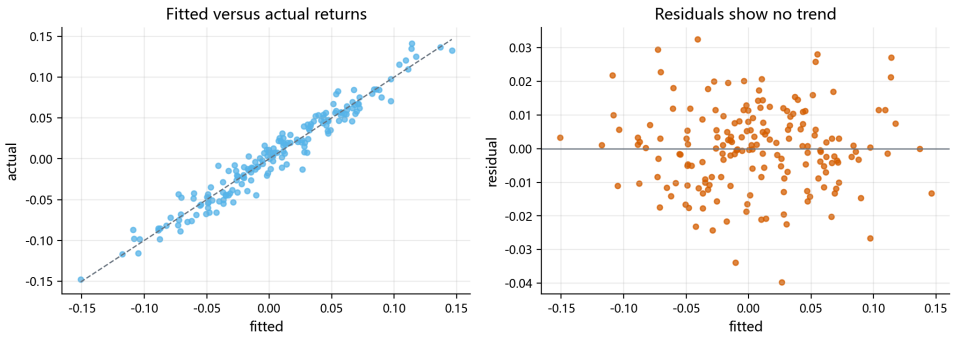
HML

High **M**inus **L**ow：高账面市值比（价值）减低（成长）的历史超额收益。暴露为正 → 组合偏价值。

这两个因子后来被扩展成五因子（加入盈利与投资），但" 从数据里提炼共同因子、再用暴露解释收益" 这套逻辑没变——这就是本讲因子分析的金融版。

来源：定义据课程讲义第 10 页；五因子据 Fama & French (2015)。

【样本 + 可视化】三因子回归：暴露、拟合与残差



合成面板（180 个月）：左：拟合值 vs 实际收益，点沿对角线分布（R^2 0.959）；右：残差没有明显趋势，说明线性设定合理。三个 β 的估计与真值非常接近（1.05 / 0.63 / -0.31，真值 1.05 / 0.62 / -0.28）。

程序 `l8_03_factor_fama.py` ·面板 `l8_ff3.csv` ·回归表 `l8_ff3_regression.csv`

因子模型的四个陷阱 Four Traps

1. 共线性：两个因子高度相关时，暴露估计不稳定、符号翻转；
2. 样本期选择：换个时间段，β 可能大变；
3. 幸存者偏差：只用" 活到今天" 的股票，会高估收益；
4. 把 α 当能力：α 显著可能是遗漏了因子。

共同点

四个陷阱都指向同一件事：因子模型是" 给定因子集" 的条件模型——因子集选错，一切解释都会错。

报告纪律：写清因子来源（自建还是公开数据）、样本期、股票池。这三点不写，结论无法复现。

来源：四个陷阱据资产定价实证文献与 L5 的偏差讨论；纪律为本讲补充。

从因子得分到感知图 From Factor Scores to a Map

同一套数学

因子得分是"每个样本在因子上的取值"；把前两个因子得分画成散点，就是一张二维定位图。PCA 的投影也是同一回事。

差别在解释

因子图要你命名两个轴（价格—性能、奢华—运动……），名字取错，图就白画了。

下一部分（帧 55）用品牌数据做一遍：12 个品牌的感知坐标、圆的大小表示份额——这就是营销课上说的perceptual map。

来源：与感知图的关系据课程讲义第 25–26 页；说明为本讲补充。

小结：因子分析是”找潜变量”

三句话

因子是看不见的共同原因；载荷告诉谁受它影响；旋转让结构可解释；得分可以画成定位图。

与 PCA 的分工

PCA 压缩数据，因子解释相关；金融里它叫三因子模型，营销里它叫感知图。



下一部分：不再" 找方向"，而是" 把样本分组"——聚类四大家族。

来源：小结为本讲整理。

聚类的定义与目标 What Clustering Optimises

定义

把样本分成若干簇 cluster，使得：

- 簇内样本彼此相似——簇内凝聚 cohesion；
- 簇间样本彼此不相似——簇间分离 separation。

问题

这两个性质都依赖距离的定义 (帧 9) 与簇形状的假设 (帧 13)。换距离或换假设，"好聚类" 就变了。

课程讲义的表述："a cluster contains a subset of observations such that all the observations within the same cluster are deemed similar."——注意"deemed"：相似是人定义的。

来源：定义与两个性质据课程讲义第 19 页；"deemed" 的提醒为本讲补充。

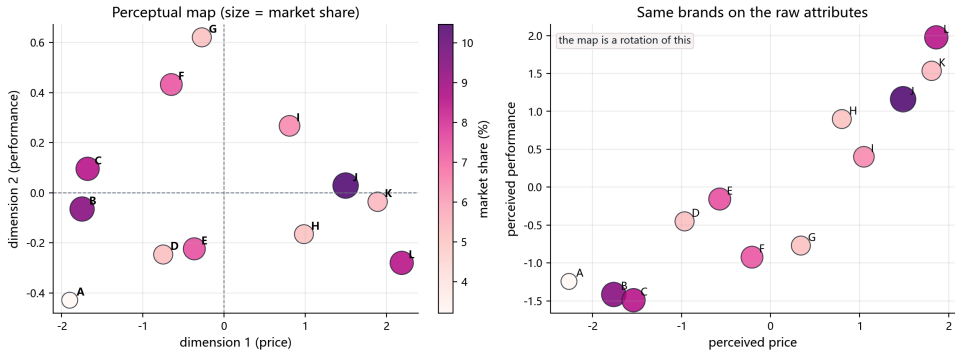
聚类的四个问题 Four Questions, Four Answers

问题	说明	谁给出答案
簇形状	凸？任意？椭圆？	由算法家族的假设决定
簇个数	事先给定还是自动发现？	k-means 要 k；DBSCAN 自动
噪声	有没有"不属于任何簇"的点？	密度型有；质心型没有
可扩展性	$O(n)$ 还是 $O(n^2)$？	层次聚类最贵

回答这四个问题，就能把十种聚类算法排成一张表（帧 12）。项目里选算法，本质是回答"我的数据有噪声吗？簇大概什么形状？k 我知道吗？"

来源：四个问题据课程讲义第 20–23 页；表述为本讲组织。

【样本 + 可视化】感知图：12 个品牌的定位



合成品牌数据（12 品牌 × 260 位受访者）：左：两个可解释维度上的定位，圆的大小是市场份额，颜色深浅也是份额；右：同样的品牌画在原始属性上——两者只差一个旋转。定位图的价值在于" 把竞品关系一眼说清"。

程序 18_04_perceptual_map.py ·样本 18_brands.csv ·坐标 18_perceptual_coords.csv

定位图怎么读、怎么误读 Reading a Positioning Map

能回答的问题

- 我们的品牌最靠近谁？（直接竞品）
- 哪个区域是空白的？（机会）
- 价格与性能的权衡线上，我们偏哪端？

三个误读

1. 把轴的方向当结论（轴是旋转出来的，符号无意义）；
2. 用少量受访者就下结论（要报样本量与稳定性）；
3. 把"感知"当"事实"（顾客觉得贵 $\neq$ 真的贵）。

课程讲义强调这是消费者感知的地图，不是产品参数的对比表。两者的差距本身就是一个有趣的业务问题。

来源：读法据营销定位实践；三个误读据课程讲义第 25–26 页与本讲补充。

【代码】k-means：肘部法与轮廓系数

```python
from sklearn.cluster import KMeans                    # the algorithm
from sklearn.metrics import silhouette_score          # an internal metric
for k in range(1, 11):                                # sweep the number of clusters
    model = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X) # fit with ten restarts
    silhouette = float(silhouette_score(X, model.labels_)) if k > 1 else float("nan") # needs k >= 2
    rows.append({"k": k, "inertia": round(float(model.inertia_), 2), # within-cluster sum of squares
                 "silhouette": round(silhouette, 4) if k > 1 else None}) # silhouette coefficient
best_k_sil = int(sweep.loc[sweep["silhouette"].idxmax(), "k"]) # k by silhouette
labels = fitted[k].predict(grid).reshape(XX.shape)    # cell labels on the grid
ax.contourf(XX, YY, labels, levels=np.arange(-0.5, k + 0.5, 1), alpha=0.22, cmap="RdPu") # Voronoi cells
ax.scatter(fitted[k].cluster_centers_[:, 0], fitted[k].cluster_centers_[:, 1], s=120, # centres
           marker="X", color="#1E2630", label="centres") # marker style
```

实测（本讲四簇样本）：惯性从 $k=1$ 到 $k=4$ 快速下降，$k=4$ 后变缓；而轮廓系数偏好 $k=2$——两个准则给了不同答案（帧 67 讨论怎么办）。完整程序 `18_05_kmeans.py`。

k-means 在优化什么 The k-means Objective

目标

$$\min_{C_1..C_k} \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2,$$

其中 μ_j 是簇 C_j 的质心 centroid。

交替求解（Lloyd 算法）

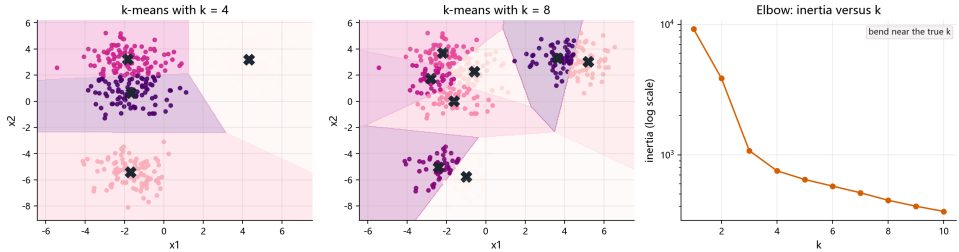
1. 分配：每个点归到最近的质心；
2. 更新：质心移到簇内均值。

反复直到不再变化。

两个后果：(1) 结果依赖初始质心（所以要 `n_init` 多次重启）；(2) 目标函数非凸，只保证局部最优。

来源：目标函数与 Lloyd 算法据 ESL 第 14 章；后果为本讲补充。

【样本 + 可视化】k-means 的 Voronoi 单元与失效形状



左：$k = 4$（与真实簇数一致）时的Voronoi 单元；中：$k = 8$ 时把真正的簇切碎；右：惯性随 k 下降的肘部。课程讲义的批评在这里可见：k-means 假设等大小、凸的簇（源稿第 21 页）。

程序 `l8_05_kmeans.py` ·样本 `l8_blobs.csv` ·扫描表 `l8_kmeans_sweep.csv`

k-means 的四个失效场景 Four Failure Modes

1) 非球形	2) 大小悬殊	3) 密度不同	4) 离群点
同心环、月牙形被切成两半。	大簇被拆开去" 平衡" 小簇。	稀疏簇被并入邻近的密簇。	少数远点各自占一个质心。

四条都来自同一个假设：簇是" 等大小、凸、密度相近" 的球。这就是为什么帧 13 说" 选算法 = 选你认为簇长什么样"。

来源：失效模式据课程讲义第 21 页与 ESL 第 14 章；图示见程序 l8_05。

层次聚类：凝聚与分裂 Hierarchical Clustering

两种策略

- 凝聚式 agglomerative（自下而上）：每个点自成一簇，逐步合并；
- 分裂式 divisive（自上而下）：一个大簇逐步切开。

实践里凝聚式更常用。

簇间距离（linkage）

- 单链 single：最近点对；
- 全链 complete：最远点对；
- 平均 average：所有点对均值；
- Ward：合并后簇内方差增量最小。

课程讲义的表述：连通型聚类基于" 物体与近处物体更相关" 这一核心思想——所以它对链式结构特别敏感（帧 64）。

来源：两种策略与 linkage 据课程讲义第 20、27 页与 ESL。

树状图怎么读 Reading a Dendrogram

三个要素

- 纵轴：合并距离（越高越" 难合并"）；
- 横轴：样本顺序（可任意旋转）；
- 切一刀：在某个高度切，得到若干个簇。

怎么定切点

找最长的垂直线段，在它中间切；或者按业务需要的簇数切，再看合并距离是否" 合理"。

层次聚类的优势：一次拟合给出整条路径（从 1 个簇到 n 个簇），可以事后选切点。代价是 $O(n^2)$ 内存，样本大就跑不动。

来源：读法据 ESL 与课程讲义第 27 页；代价说明为本讲补充。

【代码】层次聚类与树状图

```python
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster # the scipy hierarchy tools
methods = ["single", "complete", "average", "ward"]   # the four classic linkages
for method in methods:                                 # loop over the linkages
    Z = linkage(X, method=method)                      # build the hierarchy
    labels = fcluster(Z, t=2, criterion="maxclust")    # cut into two clusters
    ari = adjusted_rand_score(true_labels, labels)     # agreement with the truth
    sizes = sorted(np.bincount(labels)[1:].tolist(), reverse=True) # cluster sizes
    rows.append({"linkage": method, "ari": round(float(ari), 4), # agreement
                 "largest_cluster": int(max(sizes)),   # size of the biggest cluster
                 "clusters": int(len(set(labels)))})   # cluster count after the cut
Z = linkage(X, method=method)                          # rebuild the hierarchy
dendrogram(Z, ax=ax, no_labels=True, color_threshold=0, # draw the tree
           above_threshold_color="#66717D", link_color_func=lambda k: "#8A0C3C") # red branches
agg = AgglomerativeClustering(n_clusters=4, linkage="ward").fit(Xb) # Ward linkage on blobs
```

实测：在"两簇 + 桥"的样本上，**单链的 ARI 最低**（被桥粘住），Ward 与全链更接近真实分组；四簇样本上 Ward 的 ARI 为 0.897。完整程序 `18_06_hierarchical.py`。

单链接效应 Single-Link Effect

现象

两个密集簇之间只要有一条稀疏的桥，单链接就会沿桥合并，把两簇并成一个。这叫单链接效应 single-link effect。

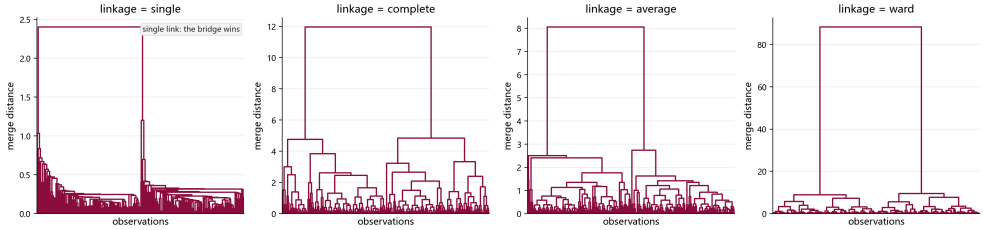
课程讲义的两张经典图（第 20 页）

- 高斯数据上 35 个簇时，最大簇开始碎裂；
- 密度簇上" 分出 20 个簇"，多数只含单个元素——因为连接式聚类没有" 噪声" 概念。

对策：用全链/Ward，或改用密度聚类（帧 68）——它有明确的噪声标签。

来源：现象与两张经典图据课程讲义第 20 页；对策为本讲补充。

【样本 + 可视化】四种 linkage 的树状图



同一份" 两簇 + 桥" 样本：单链（左一）在很低的距离就把两簇连起来，树看起来" 一脉相承"；全链与Ward（右两张）留住了一个高得多的合并距离，说明它们识别出了两簇结构。这就是"linkage 决定你看到什么"。

程序 `18_06_hierarchical.py` ·样本 `18_chain.csv` ·对照表 `18_linkage_compare.csv`

案例：用聚类做投资分散与客户分群

股票聚类（源稿第 28–29 页）

用财务指标（估值、成长、盈利能力）把股票分组：同组股票特征相似、跨组相关性低，于是每组挑一两只就能分散风险。

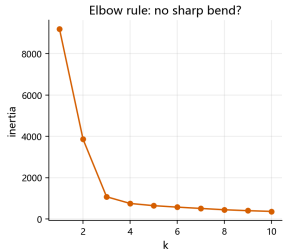
客户分群（源稿第 30 页）

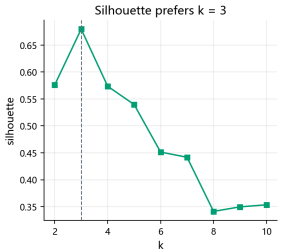
用 RFM/行为指标把客户分组：每组的营销动作不同。案例清单里有 Starbucks 顾客分群的完整流程。

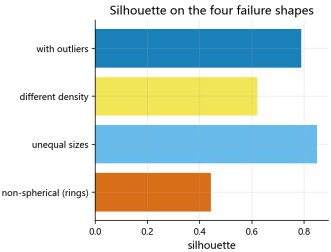
两个案例的共同点：分群本身不是目的，目的是" 分组之后做什么"。所以评估要看下游动作的效果，不只看轮廓系数。

来源：案例据课程讲义第 28–30 页；评估提醒为本讲补充。

【样本 + 可视化】k 的选择：肘部与轮廓给出不同答案







左：惯性曲线——$k = 4$ 之后变缓（肘部指向 4）；中：轮廓系数在 $k = 2$ 最高；右：四种失效形状上的轮廓值。两个准则冲突时怎么办：看下游任务、看稳定性、看业务是否接受。

程序 `18_05_kmeans.py` · 曲线表 `18_kmeans_sweep.csv` · 失效表 `18_kmeans_failures.csv`

密度聚类：DBSCAN 的定义 Density-Based Clustering

两个参数

- ε：邻域半径；
- minPts：邻域内至少多少个点。

三类点

- 核心点 core point：邻域内点数 $\geq$ minPts；
- 边界点 border point：在核心点邻域内但自身不是核心；
- 噪声点 noise point：两者都不是。

优点

不需要指定 k；能发现任意形状；自带噪声标签。

缺点

密度不均时两个参数很难同时合适；高维下距离失效（L7 帧 51）。

来源：定义据课程讲义第 23 页与 scikit-learn 文档。

【代码】DBSCAN 与 HDBSCAN

```python
from sklearn.cluster import DBSCAN, HDBSCAN, KMeans  # the three methods
for eps in [0.3, 0.6, 0.9, 1.3]:                     # neighbourhood radius
    for min_pts in [3, 6, 12]:                       # minimum points per core
        model = DBSCAN(eps=eps, min_samples=min_pts).fit(X) # fit
        labels = model.labels_                       # -1 marks noise
        n_clusters = len(set(labels) - {-1})         # number of clusters
        noise_share = float(np.mean(labels == -1))   # share of noise points
best = DBSCAN(eps=0.9, min_samples=6).fit(X)         # a workable setting
core_mask[best.core_sample_indices_] = True          # mark the core points reported by sklearn
border_mask = (~core_mask) & (labels != -1)          # border points belong to a cluster but are not core
noise_mask = labels == -1                            # noise points
hdb = HDBSCAN(min_cluster_size=25, min_samples=8).fit(X) # HDBSCAN needs no radius
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X) # k-means must be told k
```

实测（双环 + 团块 + 均匀噪声）：$\varepsilon = 0.9$、minPts$= 6$ 时找到两个环与一个团块，噪声约 11%；HDBSCAN 自动给出 7 个簇；k-means（$k = 3$）把环切成两半。完整程序 18_07_dbscan.py。

分布聚类：高斯混合与 EM Gaussian Mixtures

模型

假设数据由 K 个高斯混合而成：

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k).$$

每个簇有自己的均值与协方差（可以斜、可以扁）。

EM 算法

1. E 步：算每个点属于各簇的责任度 responsibility（后验概率）；
2. M 步：用责任度加权更新 π, μ, Σ。

每轮都单调提高对数似然（帧 72 实测）。

与 k-means 的关系：k-means 是" 协方差相同且各向同性、责任度取 $0/1$" 的特例。所以 GMM 更灵活，也给软分配。

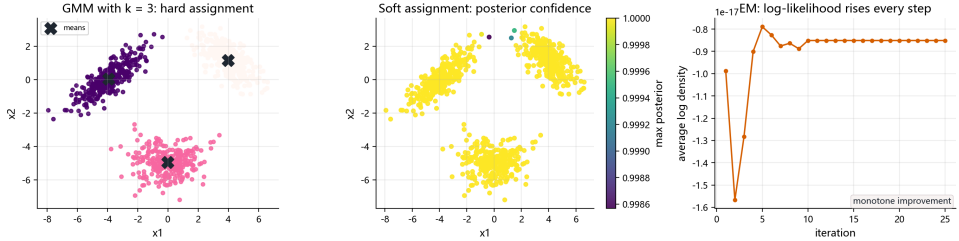
来源：模型与 EM 据 ESL 第 8 章与课程讲义第 22 页。

【代码】GMM 与 EM 迭代

```python
from sklearn.mixture import GaussianMixture          # the EM implementation
for k in [2, 3, 4]:                                  # under, correct and over
    gmm = GaussianMixture(n_components=k, covariance_type="full", random_state=42).fit(X) # full covariance
    labels = gmm.predict(X)                          # hard assignment from the posteriors
    rows.append({"components": k, "bic": round(float(gmm.bic(X)), 1), # BIC: lower is better
                 "log_likelihood": round(float(gmm.score(X) * len(X)), 1), # total log-likelihood
                 "ari_vs_truth": round(float(adjusted_rand_score(truth, labels)), 4)}) # agreement
best_k = int(comparison.loc[comparison["bic"].idxmin(), "components"]) # k by BIC
means = Xm[np.random.default_rng(42).choice(len(Xm), k, replace=False)] # initialise on random points
resp[:, j] = weights[j] * norm * np.exp(-0.5 * np.einsum("ni,ij,nj->n", diff, inv, diff)) # density
resp /= np.maximum(resp.sum(axis=1, keepdims=True), 1e-300) # normalise the responsibilities
weights = nk / nk.sum()                              # M step: weights
means = (resp.T @ Xm) / nk[:, None]                  # M step: means
post = gmm3.predict_proba(X)                         # soft assignment
```

实测：$k = 3$ 的 BIC 最小（与真实成分数一致）；手写 EM 的平均对数密度从 -4.6 单调升到 -3.3。完整程序 `18_08_gmm_em.py`。

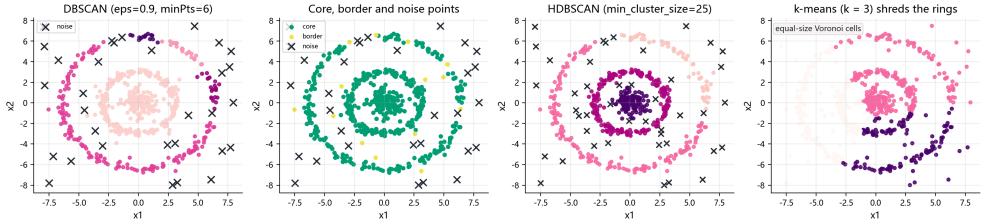
【样本 + 可视化】GMM 的软分配与 EM 轨迹



三个倾斜的椭圆高斯：左：硬分配（取后验最大）；中：软分配——颜色越亮表示越"确定"，边界处自然模糊；右：手写 EM 的对数密度曲线，每一步都不下降（这是 EM 的保证）。

程序 `18_08_gmm_em.py` ·样本 `18_gmm.csv` ·迭代表 `18_em_trace.csv`

【样本 + 可视化】DBSCAN 的密度结构、噪声与 k-means 的失败



双环 + 团块 + 均匀噪声：左：DBSCAN 找回两环与一团块，并把背景标为噪声；中：核心/边界/噪声三类点；右二：HDBSCAN 自动分簇；右一：k-means（$k = 3$）把环切断——这正是帧 60 的" 非球形失效"。

程序 `18_07_dbscan.py` ·样本 `18_density.csv` ·参数表 `18_dbscan_grid.csv`

小结：四族各答一种”簇形状”

家族	代表	用它当
连通	层次	要层次结构、n 不大
质心	k-means	凸簇、k 已知、要快
分布	GMM	要软分配、椭圆簇
密度	DBSCAN/HDBSCAN	有噪声、任意形状

下一部分：四个家族都跑完了，怎么证明分得好不好？——内部指标、外部指标、稳定性，以及异常与关联规则。

来源：小结为本讲整理。

内部指标：只看数据本身 Internal Metrics

指标	形式（直觉）	方向
轮廓系数 silhouette coefficient	$(b-a)/\max(a,b)$	越大越好，$[-1,1]$
CH 指数	簇间方差 / 簇内方差	越大越好
DB 指数	簇内散度 / 簇间距离	越小越好

三个指标都在回答" 紧不紧、离得远不远"，但对簇形状的偏好不同：轮廓偏好凸与球形，所以它对密度型/环状簇会给出低分——低分不等于" 聚类错"。

来源：指标定义据 scikit-learn clustering metrics 文档与 ESL；偏好说明为本讲补充。

外部指标：和真标签比 External Metrics

两个常用

- 调整兰德指数 adjusted Rand index (ARI)：成对一致的比例，随机分组的期望为 0；
- 标准化互信息 normalized mutual information (NMI)：两个划分共享多少信息。

”调整”的意义

普通的 Rand 指数在随机分组时并不为 0（簇越多越容易”碰巧一致”），调整版减掉了随机期望——所以才能跨 k 比较。

外部指标的用处：有真标签时（如已知客户真实分层）评估聚类；或者用它检验”算法能否还原已知结构”（本讲帧 80）。业务里通常没有真标签。

来源：指标与调整的含义据 scikit-learn 文档与 ESL；用途说明为本讲补充。

内部指标什么时候骗人 When Internal Metrics Mislead

三种骗法

1. 偏好凸簇：环状/密度簇得分低，但分组其实更好；
2. 不同 k 之间不可比：轮廓系数天然偏向小 k（簇少、分离大）；
3. 受噪声影响：噪声点拉低轮廓，但删掉又会丢信息。

本讲实测（帧 80）

四簇样本上，轮廓系数在 $k = 2$ 最高（0.663），而真实簇数是 4（$k = 4$ 时 0.578）——内部指标把"错的分组"判成了更好。

所以无监督结论必须多准则 + 下游验证：内部指标只是"体检的一管血"，不是"诊断书"。

来源：三种骗法据 ESL 与实践；实测数字来自程序 l8_09。

稳定性：最被低估的检验 Stability

做法

重复 B 次：重抽样（bootstrap 或子采样）$\rightarrow$ 重新聚类 $\rightarrow$ 算两次划分之间的ARI。稳定的话 ARI 应接近 1。

怎么用

- 比较不同 k：选 ARI 最高且稳定的；
- 比较不同算法：稳定的那个更可信；
- 报告里给出"重抽样 100 次，平均 ARI $=$ 0.87"。

稳定性检验能挡住最常见的失败：把噪声分成了"群体"。一条数据上的漂亮分群，重抽样后可能完全变样。

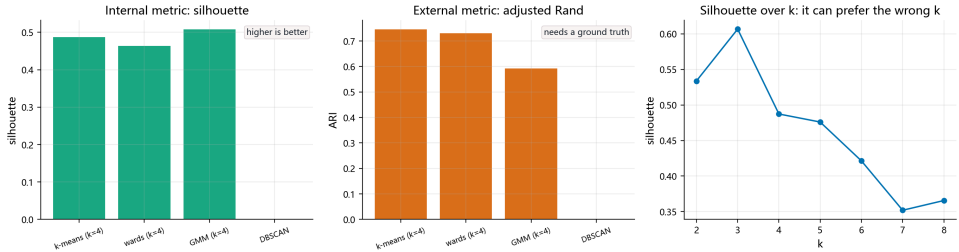
来源：稳定性检验据 Hennig (2007) 与聚类实践；报告写法为本讲建议。

【代码】聚类评估：内部与外部指标

```python
from sklearn.metrics import (silhouette_score, calinski_harabasz_score, # internal metrics
                             davies_bouldin_score, adjusted_rand_score,
                             normalized_mutual_info_score) # external metrics
for name, model in runs.items():                          # loop over the four families
    labels = model.labels_ if hasattr(model, "labels_") else model.predict(X) # labels from the model
    mask = labels != -1                                   # ignore the noise label for the metrics
    sil = float(silhouette_score(X[mask], labels[mask])) if len(set(labels[mask])) > 1 else float("nan") # silhouette
    ch = float(calinski_harabasz_score(X[mask], labels[mask])) if len(set(labels[mask])) > 1 else float("nan") # CH
    db = float(davies_bouldin_score(X[mask], labels[mask])) if len(set(labels[mask])) > 1 else float("nan") # DB
    ari = float(adjusted_rand_score(truth[mask], labels[mask])) # external: adjusted Rand
    nmi = float(normalized_mutual_info_score(truth[mask], labels[mask])) # external: mutual information
for k in range(1, 9):                                     # sweep k
    model = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X) # fit
    sil = float(silhouette_score(X, model.labels_)) if k > 1 else float("nan") # silhouette needs k >= 2
```

实测：四种算法在同一份四簇样本上，k-means 与 GMM 的 ARI 最高（约 0.9），DBSCAN 略低；而轮廓系数在 $k = 2$ 反常地最高。完整程序 `18_09_metrics.py`。

【样本 + 可视化】四种算法、两种指标，没有唯一最优



左：四种算法的轮廓系数（内部）；中：调整兰德指数（外部，需要真标签）；右：轮廓系数随 k 的变化——$k = 2$ 最高，而真实簇数是 4。这张图就是帧 77 的实证：内部指标不能当真理。

程序 `18_09_metrics.py` ·指标表 `18_cluster_metrics.csv` ·陷阱表 `18_metric_trap.csv`

异常检测的四种思路 Four Roads to "Unusual"

思路	怎么定义异常	代表方法
距离	离所有点都远	kNN 距离、LOF
密度	局部密度明显低	LOF、DBSCAN 的噪声标签
重构	用低维表示重构误差大	PCA 重构误差、自编码器
隔离	很少几刀就能被" 孤立"	孤立森林 isolation forest

没有标签时，" 异常" 必须先有定义——四种思路就是四种定义。报告里要写清" 我按什么定义异常"，否则阈值无法解释。

来源：四种思路据 ESL 与异常检测实践；报告要求为本讲建议。

孤立森林与 LOF Isolation Forest and LOF

孤立森林

随机选特征、随机切分，异常点平均路径更短(更容易被单独切出来)。优点：$O(n \log n)$、无需距离、对高维较稳。

LOF

比较某点与其邻居的局部密度：密度明显更低 $\rightarrow$ 异常。优点：能发现" 局部异常"；缺点：对 k 敏感。

两者的共同点：都输出分数而非标签；`contamination` 只是把分数转成标签的阈值。业务上应先看分数分布，再定阈值（与 L6/L7 的阈值讨论同理）。

来源：两种方法据 Liu et al. (2008) 与 Breunig et al. (2000)；实践提醒为本讲补充。

【代码】异常检测与关联规则

```python
from sklearn.ensemble import IsolationForest          # trees based anomaly detector
from sklearn.neighbors import LocalOutlierFactor      # density based anomaly detector
iso = IsolationForest(contamination=0.05, random_state=42).fit(X) # isolation forest
lof = LocalOutlierFactor(contamination=0.05, novelty=False).fit(X) # local outlier factor
iso_flag = (iso.predict(X) == -1).astype(int)         # flag from the forest
lof_flag = (lof.fit_predict(X) == -1).astype(int)     # flag from LOF
def support(itemset):                                 # support of an itemset
    target = set(itemset)                             # the set we test
    return sum(1 for b in baskets if target.issubset(set(b))) / len(baskets) # share of baskets
for combo in combinations(items, size):               # enumerate the combinations
    s = support(combo)                                # compute the support
    if s >= min_support:                              # keep it when frequent enough
        frequent.append((combo, s))                   # store it
confidence = s_itemset / s_a                          # confidence of a -> b
lift = confidence / support([b])                      # lift of the rule
```

实测：孤立森林与 LOF 都能以 5% 的预算抓住大部分人造异常；关联规则里最强的是 soda → beer（置信度 0.875、提升度 2.04）——正是生成数据时埋下的"派对"模式。完整程序 `18_10_anomaly_rules.py`。

关联规则：支持度、置信度、提升度

三个数

- 支持度 support：$A \cup B$ 出现在多少比例的篮子里；
- 置信度 confidence：买了 A 的人里有多少也买 B；
- 提升度 lift：置信度 $\div$ B 的无条件概率。

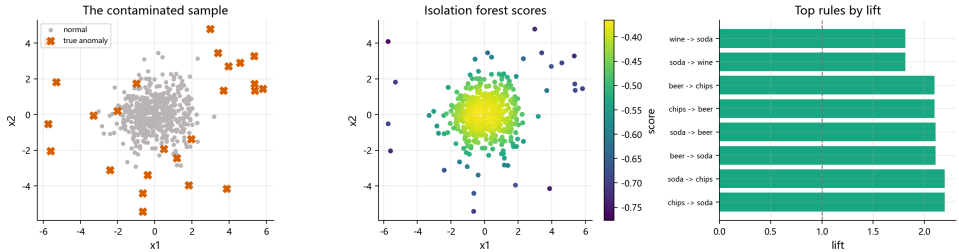
怎么读

提升度 $= 1$ 表示独立，> 1 表示正相关。只看置信度高会骗人——热销品的置信度天然就高，所以要看提升度。

算法：Apriori 用" 频繁项集的子集也频繁" 逐层筛；FP-Growth 用树结构加速。规则一多就没法看，通常按提升度排序取前 10–20 条。

来源：三个指标与算法据 Agrawal & Srikant (1994) 与课程讲义；阅读建议为本讲补充。

【样本 + 可视化】异常分数与最强规则



左：被污染的样本（灰 = 正常，叉 = 真异常）；中：孤立森林的分数——越暗越异常，能看出" 离群程度" 而不是硬标签；右：提升度最高的几条关联规则，虚线是" 独立" 参考线（lift=1）。

程序 `l8_10_anomaly_rules.py` ·样本 `l8_anomaly.csv` ·规则表 `l8_rules.csv`

小结：评估是”讲故事”的一部分

三句话

内部指标可以骗人；外部指标需要真标签；稳定性检验最被低估。异常与规则都要先定义再定价。

报告模板

1. 我假设的簇形状与距离；
2. 我用的两个准则 + 稳定性结果；
3. 分群之后做了什么、效果如何。

下一部分：2026 的” 嵌入与检索”——t-SNE/UMAP 的可视化、向量库的召回-延迟权衡、以及从 LDA 到 BERTopic 的主题建模。

来源：小结为本讲整理。

t-SNE 与 UMAP：非线性嵌入 Nonlinear Embeddings

t-SNE

把高维的"相似度"（高斯核）与低维的"相似度"（t 分布）对齐；靠困惑度 perplexity 控制有效邻居数。局部保得好，全局距离不可信。

UMAP

用"模糊拓扑"近似流形，再用低维布局去贴；更快、全局结构保留更好，参数是 `n_neighbors` 与 `min_dist`。

本讲实测（帧 90）：邻域保持度 PCA 0.052、t-SNE 0.253、UMAP 0.182——t-SNE 在这份数据上局部保得最好，但三者的二维图都不能当证据（帧 88）。

来源：方法据 van der Maaten & Hinton (2008) 与 McInnes et al. (2018)；实测来自程序 l8_11。

嵌入可视化的四个坑 Four Traps in Embedding Plots

1. 簇间距离不可比：图上离得远不代表真的不同；
2. 簇的大小无意义：t-SNE 常把大簇画小、小簇画大；
3. 随机种子敏感：换个种子图就变，别盯着一次结果讲故事；
4. 过度解读：二维图是" 投影"，不是" 真相"。

怎么用才对

用图发现线索，用指标（邻域保持、ARI）与下游任务验证结论。一句话：图用来提问，不用来证明。

这也是评分表" 结果与验证" 里最容易失分的地方：一张漂亮的 t-SNE 图配上" 我们发现了 5 个群体"，但没有任何稳定性检验。

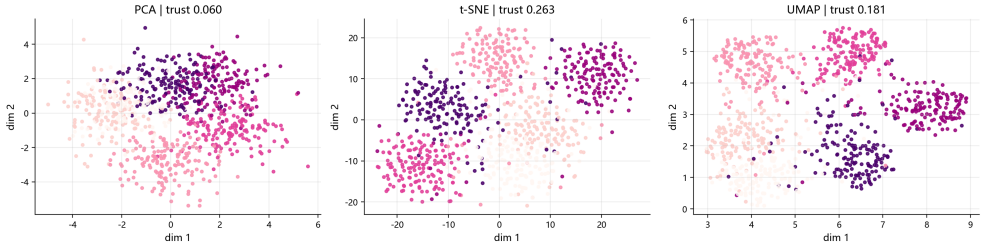
来源：四个坑据 t-SNE/UMAP 文献与实践；用法建议为本讲补充。

【代码】PCA / t-SNE / UMAP 三联对照

```python
from sklearn.decomposition import PCA                # the linear baseline
from sklearn.manifold import TSNE                    # t-SNE
from sklearn.neighbors import NearestNeighbors       # neighbourhood preservation
pca = PCA(n_components=2, random_state=42).fit_transform(X) # linear projection
tsne = TSNE(n_components=2, perplexity=30, init="pca", random_state=42).fit_transform(X) # t-SNE
umap_emb = UMAP(n_neighbors=15, min_dist=0.1, random_state=42).fit_transform(X) # UMAP
high = NearestNeighbors(n_neighbors=k + 1).fit(X).kneighbors(X, return_distance=False)[:, 1:] # high-dim neighbours
low = NearestNeighbors(n_neighbors=k + 1).fit(embedding).kneighbors(embedding, return_distance=False)[:, 1:] # low-dim neighbours
keep += len(set(high[i]).intersection(set(low[i])))   # overlap of the two neighbourhoods
return keep / (len(X) * k)                           # share preserved
for ax, (name, coords) in zip(axes, embeddings.items()): # loop over the projections
    sc = ax.scatter(coords[:, 0], coords[:, 1], c=labels, cmap="RdPu", s=10, alpha=0.85) # colour by group
```

实测（900 行 × 40 维 × 6 组）：邻域保持度 PCA 0.052、t-SNE 0.253、UMAP 0.182。注意：邻域保持高只说明"邻居还在旁边"，不说明簇的个数对。完整程序 `18_11_embeddings.py`。

【样本 + 可视化】嵌入空间：三种投影与检索结果



同一份 40 维样本的三种二维投影：PCA 把六个组压成一团（线性不够用）；t-SNE 把六组分开但组间距离不可信；UMAP 介于两者之间。标题里的 trust 是邻域保持度——它是可量化的，图本身不是。

程序 `18_11_embeddings.py` ·样本 `18_highdim.csv` ·对照表 `18_embedding_compare.csv`

向量检索与近似最近邻 Approximate Nearest Neighbour

问题

库里有一千万条向量，来一个查询就要找最相似的 10 条。精确搜索是 $O(nd)$，太慢。

近似搜索的两种结构

- 倒排文件 inverted file index (IVF)：先粗聚类，只在最近的几个簇里搜；
- HNSW hierarchical navigable small world：图结构，逐层下降。

核心权衡：召回率 vs 延迟。参数（如 `nprobe`）越大越准也越慢。选型时要先定"我能接受多少召回损失"。

来源：结构据 Johnson, Douze & Jégou (2019) 与 FAISS 文档；权衡说明为本讲补充。

【代码】FAISS：精确索引 vs 倒排索引

```python
import faiss                                          # the vector search library
exact = faiss.IndexFlatL2(X.shape[1])                 # brute-force L2 index
exact.add(X)                                          # add the corpus
D_exact, I_exact = exact.search(queries, k)           # exact search
n_lists = 40                                          # number of inverted lists
quantizer = faiss.IndexFlatL2(X.shape[1])             # the coarse quantiser
ivf = faiss.IndexIVFFlat(quantizer, X.shape[1], n_lists, faiss.METRIC_L2) # inverted-file index
ivf.train(X)                                          # train the coarse quantiser
ivf.add(X)                                            # add the corpus
for nprobe in [1, 2, 4, 8, 16, 40]:                   # probes per query
    ivf.nprobe = nprobe                               # set the search breadth
    D_ann, I_ann = ivf.search(queries, k)             # approximate search
    recall = float(np.mean([len(set(I_exact[i]).intersection(set(I_ann[i]))) / k for i in range(len(queries))])) # recall at k
```

实测（4000 条 32 维向量、10 近邻）：nprobe=1 时召回 0.62、加速 10 倍以上；nprobe=16 时召回 0.97、仍有 4.2 倍加速；nprobe=40 时召回 1.00。完整程序 18_12_faiss_ann.py。

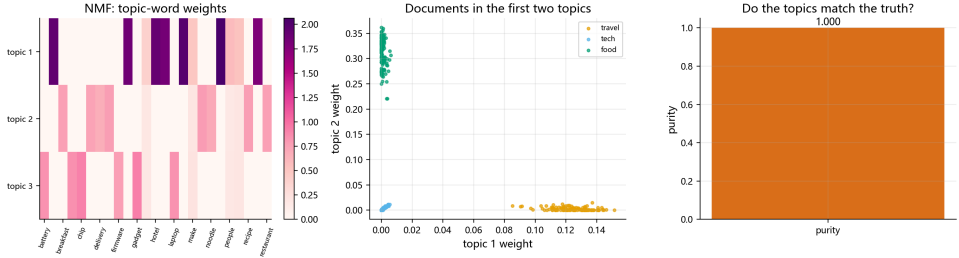
主题建模：从 LDA 到 BERTopic Topic Modelling

方法	输入	特点
潜在狄利克雷分配 latent Dirichlet allocation (LDA)	词袋	生成式、可解释；忽略词序与语义
非负矩阵分解 NMF	TF-IDF	可加的部分-整体；结果稳定
BERTopic	句向量	嵌入 → 聚类 → 主题词；语义更强

BERTopic 三步：(1) 用句向量模型把文档编码；(2) UMAP 降维 + HDBSCAN 聚类；(3) 每簇用 c-TF-IDF 选主题词。它把本讲前半部分的工具串成了一条流水线。

来源：LDA 据 Blei et al. (2003)；NMF 据 Lee & Seung (1999)；BERTopic 据 Grootendorst (2022)。

【样本 + 可视化】NMF 与 LDA：主题词与纯度的实测



合成语料（360 篇、三个主题）：左：NMF 的主题-词权重（可见三组词各自成块）；中：文档在前两个主题权重上的分布（颜色 = 真实主题）；右：纯度——把主题簇与真实主题对齐后的比例，本讲实测接近 1.0。

程序 `l8_13_topics.py` ·主题表 `l8_topics.csv` ·文档主题 `l8_doc_topics.csv`

自监督与对比学习 Self-Supervised and Contrastive Learning

自监督

没有人工标签，就从数据本身构造监督信号：预测被遮住的词、判断两段是否相邻、预测下一句……于是海量无标注文本/图片都能用来训练表示。

对比学习

同一对象的两个" 视角" 是正样本对，其他是负样本对；训练目标是拉近正对、推远负对。这直接产出可用于检索的嵌入。

这解释了一个现象：本讲的 PCA/聚类/ANN 在 2026 依然有用，但它们的输入往往已经是自监督模型产出的嵌入——无监督从" 处理原始数据" 升级成" 处理表示"。

来源：自监督与对比学习据 SimCLR/CLIP 等文献综述；与本讲的关系为本讲补充。

小结：无监督是”表示学习”的另一半

三句话

图用来提问、指标用来验证；向量检索的权衡是召回 vs 延迟；主题模型可以叠在嵌入之上。

工程顺序

先降维省成本 → 再嵌入 → 再 ANN 检索 → 必要时再加主题层。



来源：小结为本讲整理。

工具栈 Tool Stack

工具	擅长	本讲用在哪
scikit-learn	PCA、因子分析、聚类、指标、t-SNE	帧 24、45、57、63、69、71、79、89
scipy	层次聚类与树状图	帧 63
umap-learn	UMAP 投影	帧 89
faiss-cpu	近似最近邻检索	帧 92
statsmodels	因子回归（FF3）的推断	帧 45、49
BERTopic	嵌入 + 聚类 + 主题词	帧 93（原理）

选型口诀：降维与聚类用 sklearn，检索用 FAISS，主题用 NMF/LDA（轻）或 BERTopic（重），因子回归用 statsmodels 看推断。

来源：工具分工据各自官方文档与本讲实践。

术语表 A–M Glossary A–M

中文	English
调整兰德指数	adjusted Rand index (ARI)
异常检测	anomaly detection
近似最近邻	approximate nearest neighbour (ANN)
关联规则	association rules
BERTopic	BERTopic
边界点	border point
质心	centroid
聚类	clustering
簇内凝聚	cohesion
置信度	confidence
对比学习	contrastive learning
核心点	core point
协方差矩阵	covariance matrix
树状图	dendrogram
密度聚类	density-based clustering
降维	dimensionality reduction
距离度量	distance metric
特征值	eigenvalue
特征向量	eigenvector
期望最大化	expectation-maximisation (EM)
方差解释比例	explained variance ratio

中文	English
外部指标	external metric
因子分析	factor analysis
因子提取	factor extraction
因子载荷	factor loading
因子得分	factor score
高斯混合	Gaussian mixture model (GMM)
层次聚类	hierarchical clustering
HNSW	hierarchical navigable small world
增量 PCA	incremental PCA
内部指标	internal metric
倒排文件索引	inverted file index (IVF)
孤立森林	isolation forest
k 均值	k-means
核 PCA	kernel PCA
潜在狄利克雷分配	latent Dirichlet allocation (LDA)
潜在因子	latent factor
提升度	lift
连接方式	linkage
载荷	loading
局部离群因子	local outlier factor (LOF)
矩阵分解	matrix factorisation

本组 42 条，全表共 72 条（含符号、公式与一句话解释，见 `17_glossary.csv` 与站点讲义页）。

术语表 N–Z Glossary N–Z

中文	English
邻域保持	neighbourhood preservation
噪声点	noise point
非负矩阵分解	non-negative matrix factorisation (NMF)
标准化互信息	normalized mutual information (NMI)
感知图	perceptual map
困惑度	perplexity
定位	positioning
主成分分析	principal component analysis (PCA)
投影	projection
召回率	recall at k
重构误差	reconstruction error
旋转	rotation
碎石图	scree plot
自监督学习	self-supervised learning
簇间分离	separation
轮廓系数	silhouette coefficient
单链接效应	single-link effect
规模因子	size factor (SMB)
软分配	soft assignment
稳定性	stability
标准化	standardisation
支持度	support
t-SNE	t-distributed stochastic neighbour embedding
三因子模型	three-factor model
主题建模	topic modelling
UMAP	uniform manifold approximation and projection
唯一性方差	uniqueness
无监督学习	unsupervised learning
Voronoi 单元	Voronoi cell
价值因子	value factor (HML)

本组 30 条，全表共 72 条。含符号、公式与一句话解释，见 17_glossary.csv 与站点讲义页。

结语：没有标签时的三句话 Wrap-up

三句话

1. 先问要结构还是要解释（降维 vs 因子 vs 聚类）；
2. 选算法 = 选你认为簇/结构长什么样的假设；
3. 结论靠多准则 + 稳定性 + 下游任务，不靠一张图。

自检

- 能解释第一主成分由哪些变量构成吗？
- 能说出四种聚类各自的失败形状吗？
- 能区分内部指标与外部指标吗？

谢谢！问题与项目讨论：`Xieer.dai@szu.edu.cn`；本讲代码、样本与术语表见 `website/code/lecture08/`。

来源：结语为本讲整理；下讲预告据课程大纲。